

Getting Started: Execute Python Code

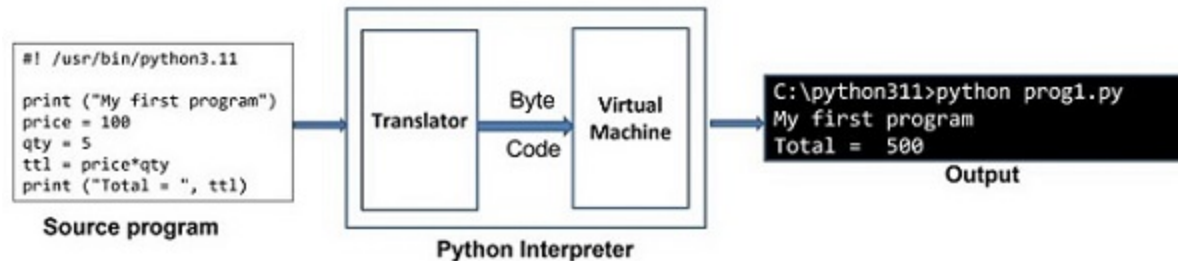
Eunji Kim

eunjikim@cau.ac.kr

OK. I have Anaconda Python interpreter on my computer... Then...

How to use Python to do something?

1. Write a Python source code for the task.
2. Execute the code using the Python interpreter and check the results.



⚠ If something goes wrong, modify the code and run it again.

Code example: `print`

```
print('hello world')
```

Execution Result:

```
hello world
```

Code example: addition

```
a = 3  
b = 2  
c = a + b  
print(c)
```

Execution Result:

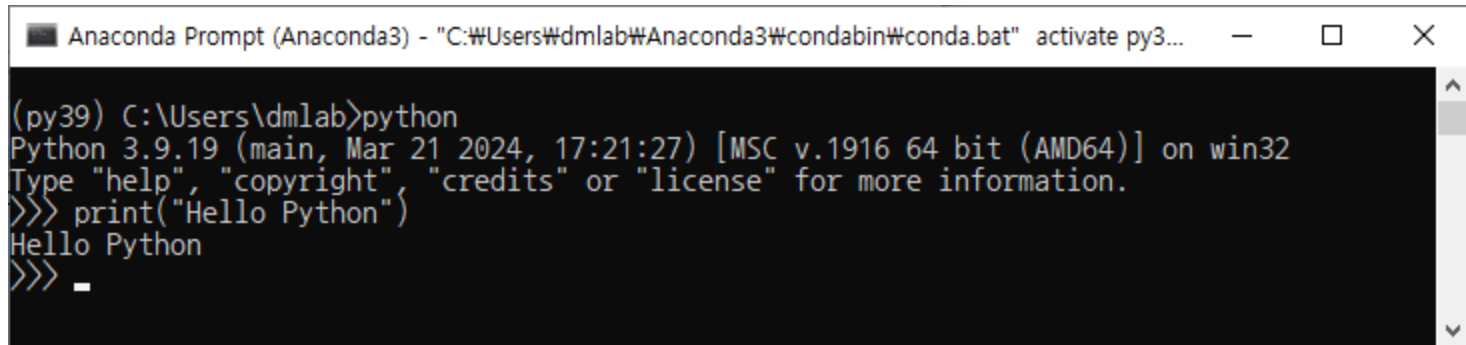
```
5
```

How to execute a python code?

We have 3 options.

1. Run inside the Python interpreter
2. Run through Python Interpreter
3. Use Jupyter notebook

1. Run inside the Python interpreter



```
Anaconda Prompt (Anaconda3) - "C:\Users\dmlab\Anaconda3\condabin\conda.bat" activate py3...  
(py39) C:\Users\dmlab>python  
Python 3.9.19 (main, Mar 21 2024, 17:21:27) [MSC v.1916 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" for more information.  
>>> print("Hello Python")  
Hello Python  
>>> _
```

Run the Python interpreter and write your code directly in the command line:

```
>>> print("Hello, World!")  
Hello, World!
```



Within `()`, enter the string you want to print enclosed in `" "` or `' '`.

2. Run through Python Interpreter

Creating a python file, using the `.py` file extension, (e.g., `mycode.py`) and running it in the Command Line:

```
C:\Users\dmlab> python mycode.py  
Hello, World!
```

For example, open a text editor and type the following code:

```
print('Hello, World!')
```

Text editor?

- ☞ If you are a Windows user, use notepad (메모장).
- ☞ If you are a MacOS user, use textEdit.

Save the text file as `mycode.py` in the directory shown on the Anaconda prompt:

```
C:\Users\dmlab>
```

For me, the location is `C:\Users\dmlab`.

To run the python code, just type `python mycode.py` on the prompt:

```
C:\Users\dmlab> python mycode.py  
Hello Python!
```


Integrated Development Environment (IDE)

What is IDE?

A code editor that bundles essential tools for writing code efficiently, such as:

- Syntax highlighting
- Debugging (finding and fixing errors)
- Intelligent code completion (suggesting code as you type)

Why Use One?

They help you organize your projects, debug efficiently, and avoid common mistakes with features that a simple text editor lacks.

Popular IDEs for Python

- **PyCharm:** A professional, feature-rich IDE (though heavy).
- **Visual Studio Code (VS Code):** A popular, lightweight, and customizable editor that works well for Python.
- **Jupyter Notebook:** An interactive, web-based environment that is perfect for data analysis, data visualization, and academic work.

For beginners, starting with a simple editor like VS Code is a great way to get comfortable before moving on to more feature-rich IDEs.

Download VS Code from <https://code.visualstudio.com/download>.

3. Jupyter Notebook

Jupyter Notebook is a web-based **interactive** development environment.



Official Jupyter Notebook Documentation

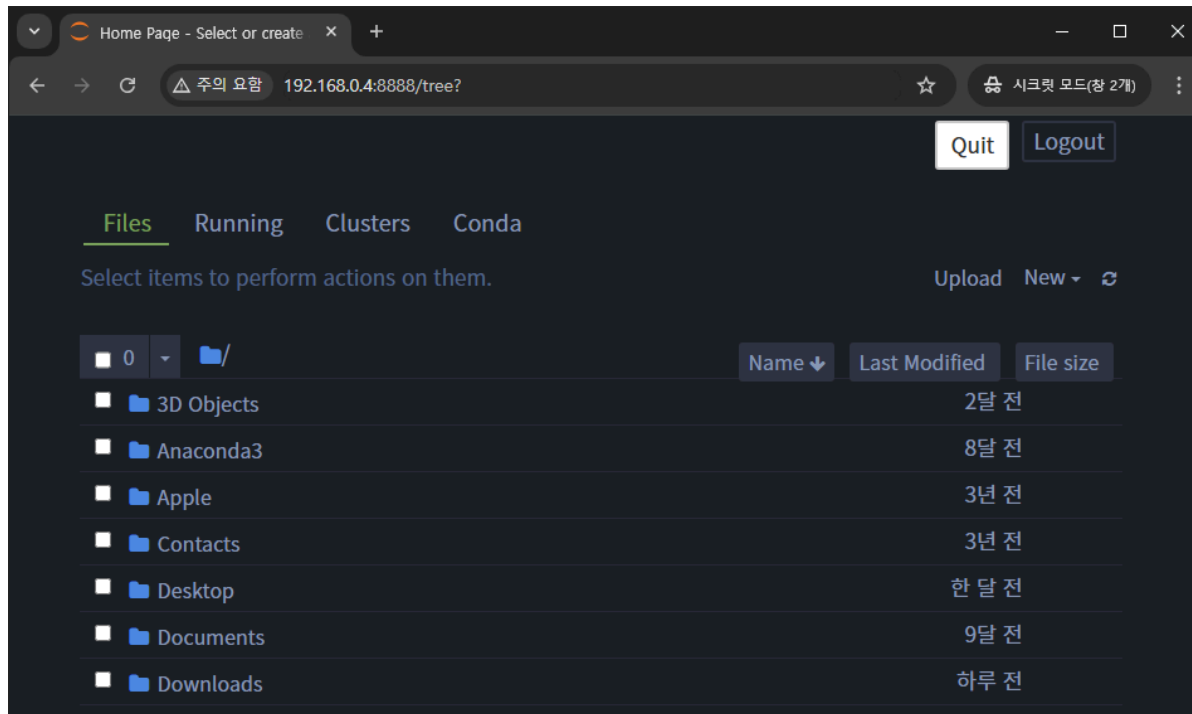
<https://jupyter-notebook.readthedocs.io/en/latest/>

To launch the Jupyter notebook, type `jupyter notebook` on the prompt:

```
C:\Users\dmlab> jupyter notebook
```

Or, simply run the `Jupyter Notebook` application bundled with Anaconda.

Then, a new tab will be opened in your default web browser that should look something like the following screenshot:

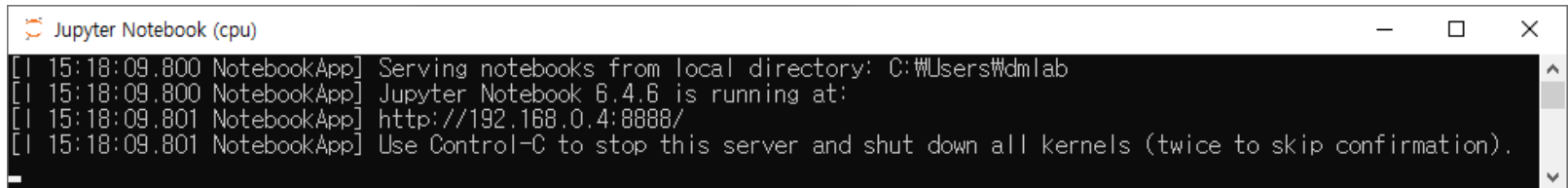


This is the **Notebook Dashboard**, specifically designed for managing your Jupyter Notebooks. It shows the directories and files located in the default working directory (`C:\Users\dm1ab` in my case).

You can traverse the directory structure to locate the folder that contains:

- the Python code (`*.py`) you would like to execute,
- the data files you would like to load (e.g., `*.csv`)
- the working directory for saving your Python notebook (`*.ipynb`).

After launching the notebook, the notebook server runs in the prompt as shown below:

A screenshot of a terminal window titled "Jupyter Notebook (cpu)". The terminal displays the following output:

```
[I 15:18:09.800 NotebookApp] Serving notebooks from local directory: C:\Users\dm1ab
[I 15:18:09.800 NotebookApp] Jupyter Notebook 6.4.6 is running at:
[I 15:18:09.801 NotebookApp] http://192.168.0.4:8888/
[I 15:18:09.801 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
```

Some information about the notebook server is printed in your terminal

- URL of the web application
 - <http://localhost:8888> by default (in my case, <https://192.168.0.4:8888>)
 - If the web browser isn't pop up, you can access to the notebook using the address shown in the server status.
- The default working directory
 - `C:\Users\dm1ab` in my case
- To stop the server, use `Ctrl-C` (twice to skip confirmation)

FAQ

⚠ If you closed the Web browser, you can reopen it using the url information in terminal. Simply drag, copy, and paste the url into your web browser.

⚠ If the URL is not accessible, check if the notebook server is still running in terminal.

⚠ [Caution] Be careful for using the `Ctrl+C` command. It may stop the Jupyter notebook server as informed in the prompt screen. If the notebook server is shut down, you cannot access the url.

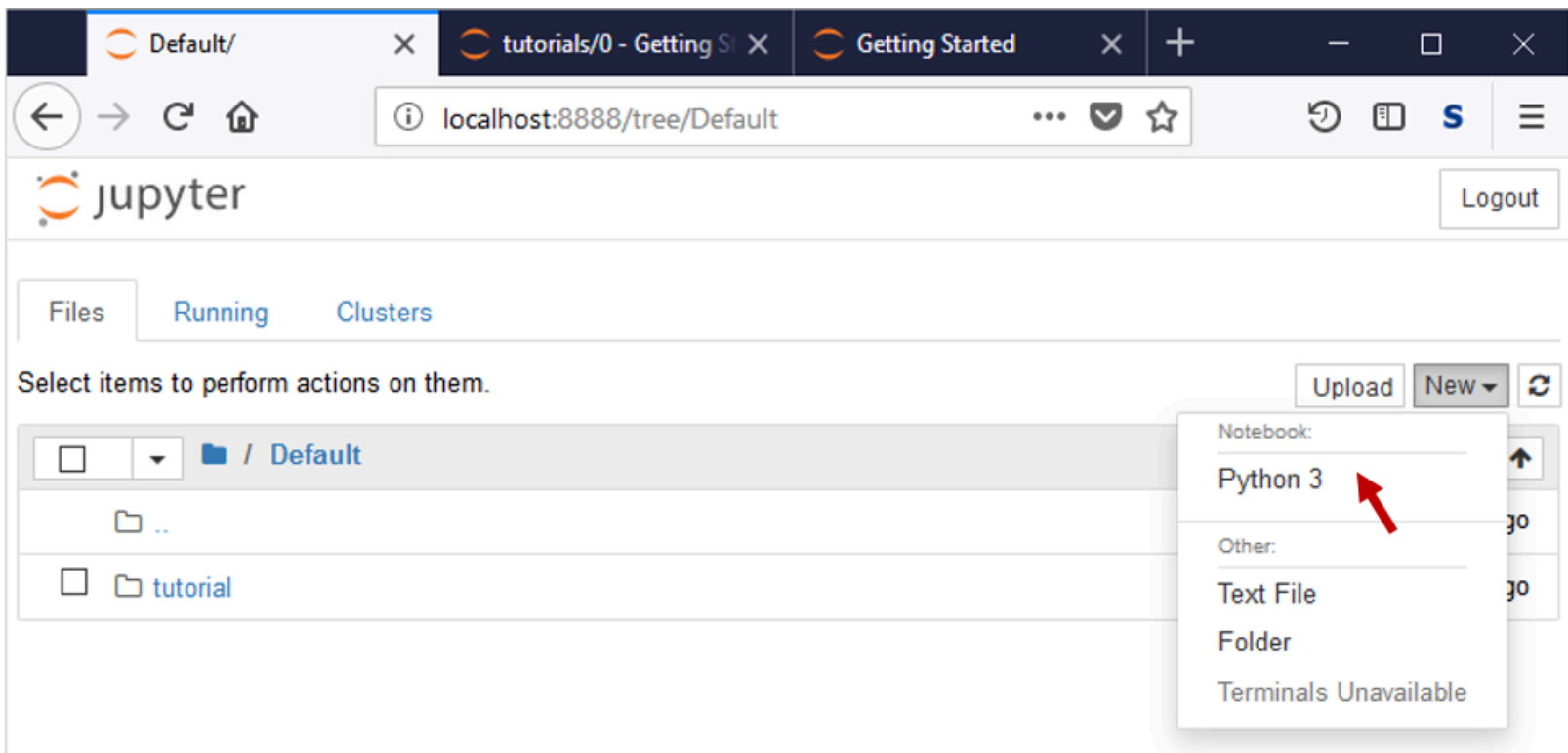
Why Jupyter Notebook?

- Web is much more comfortable than terminal (command prompt) environment.
- It is interactive and thus powerful!
- You can check the running results of codes immediately on the screen.

Creating a notebook file

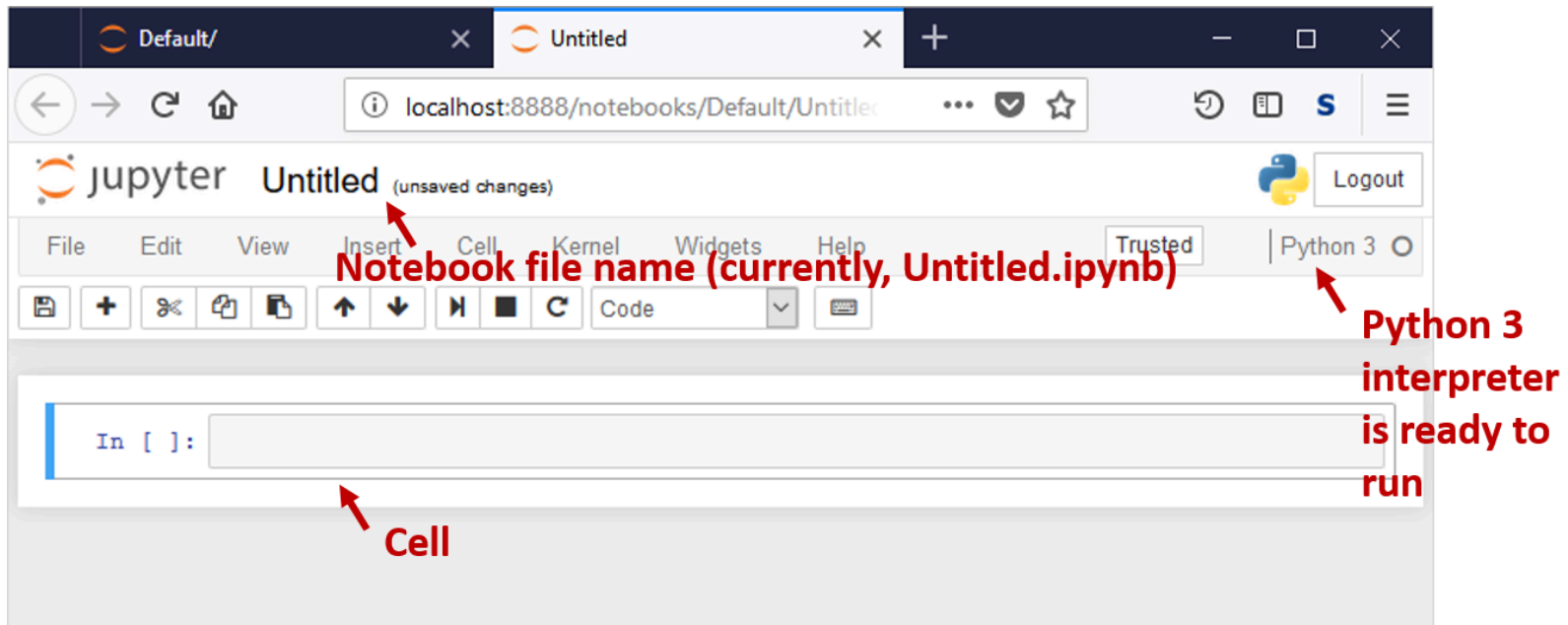
In the Jupyter Notebook environment, we will use a notebook file with a ipynb extension to write and execute Python codes (e.g., `myfirstnotebook.ipynb`).

To create a new notebook, click on the `New` button on the top right hand corner of the web page and select `Python 3` notebook.



Notebook is ready!

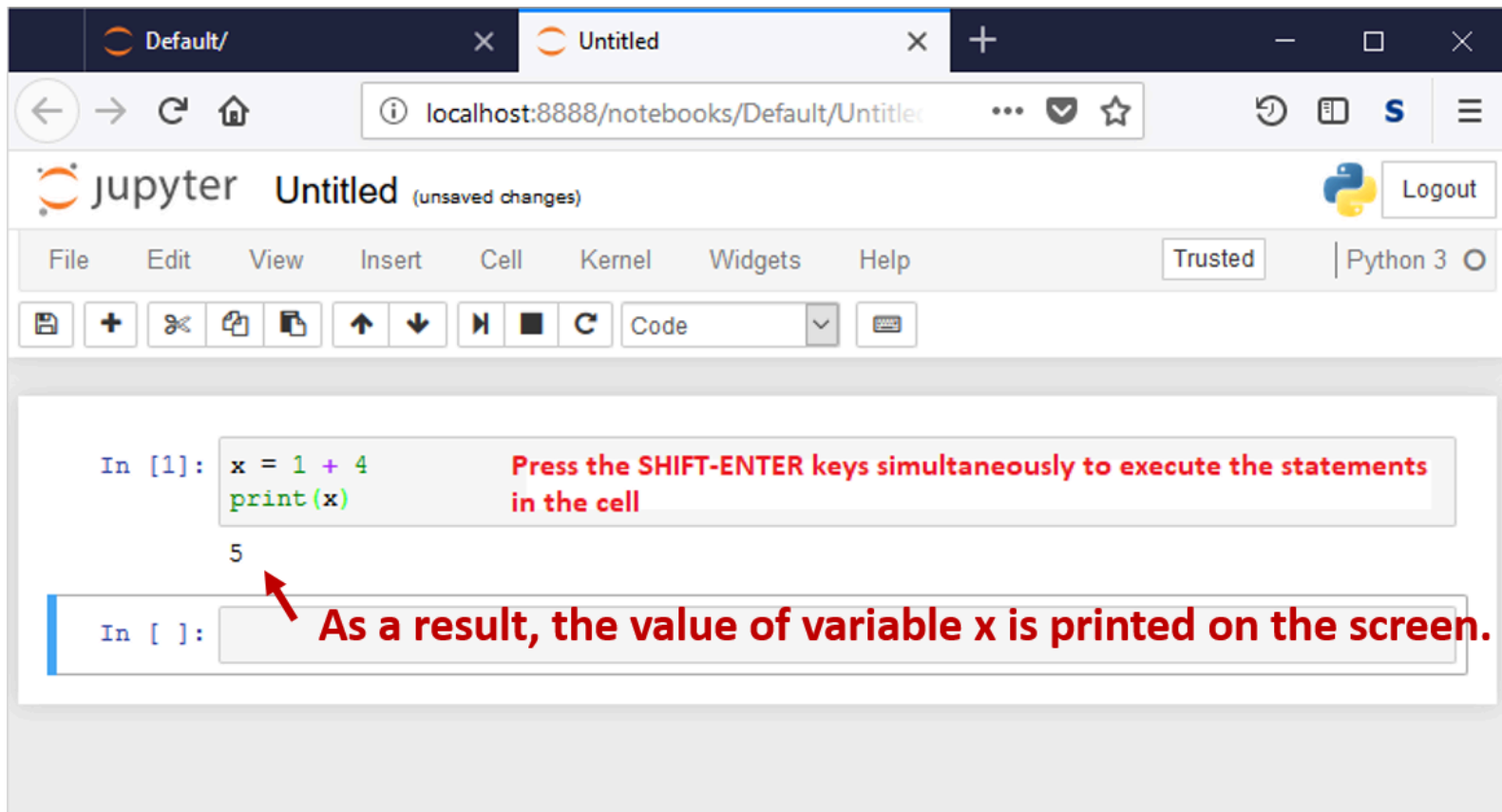
The following new web page will be rendered and the notebook is ready to accept your Python commands.



- A **cell** is like a block or a section where you can write your code or text (notes).
- The **kernel** is like the brain of the notebook. It's the "computational engine" that runs your code (here, Python 3 interpreter).

Running a code block

- To execute the Python statements within each cell, press both the **SHIFT** and **ENTER** keys simultaneously.
- The result will be displayed right below the cell, as shown in the diagram below.

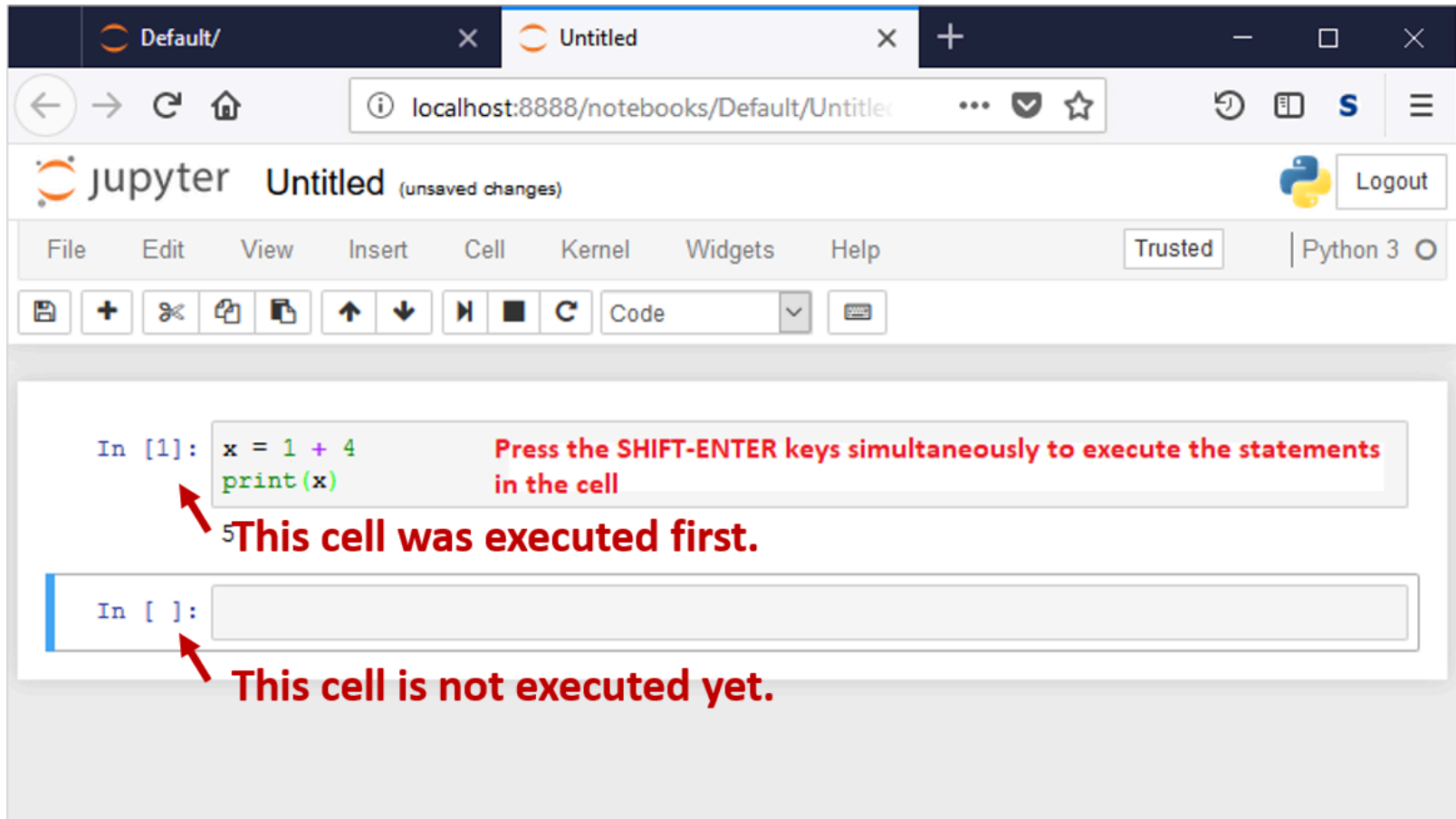


The screenshot displays a Jupyter Notebook interface. At the top, there are two tabs: 'Default/' and 'Untitled'. The browser address bar shows 'localhost:8888/notebooks/Default/Untitled'. The Jupyter logo and 'Untitled (unsaved changes)' are visible. Below the menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help), there is a toolbar with icons for saving, adding, deleting, and running code. The main area contains a code cell with the following content:

```
In [1]: x = 1 + 4  
        print(x)
```

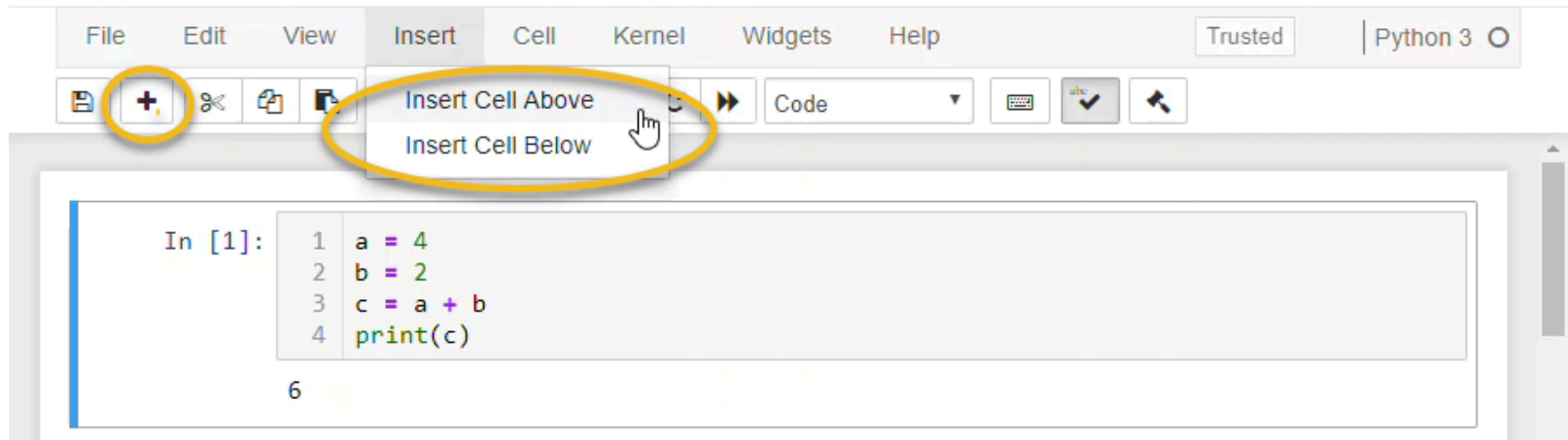
Below the code cell, the output '5' is displayed. A red arrow points from the output '5' to a red text box that says 'As a result, the value of variable x is printed on the screen.' Another red text box above the code cell says 'Press the SHIFT-ENTER keys simultaneously to execute the statements in the cell'.

The cell execution order is displayed on the left after running a cell.



Adding a cell

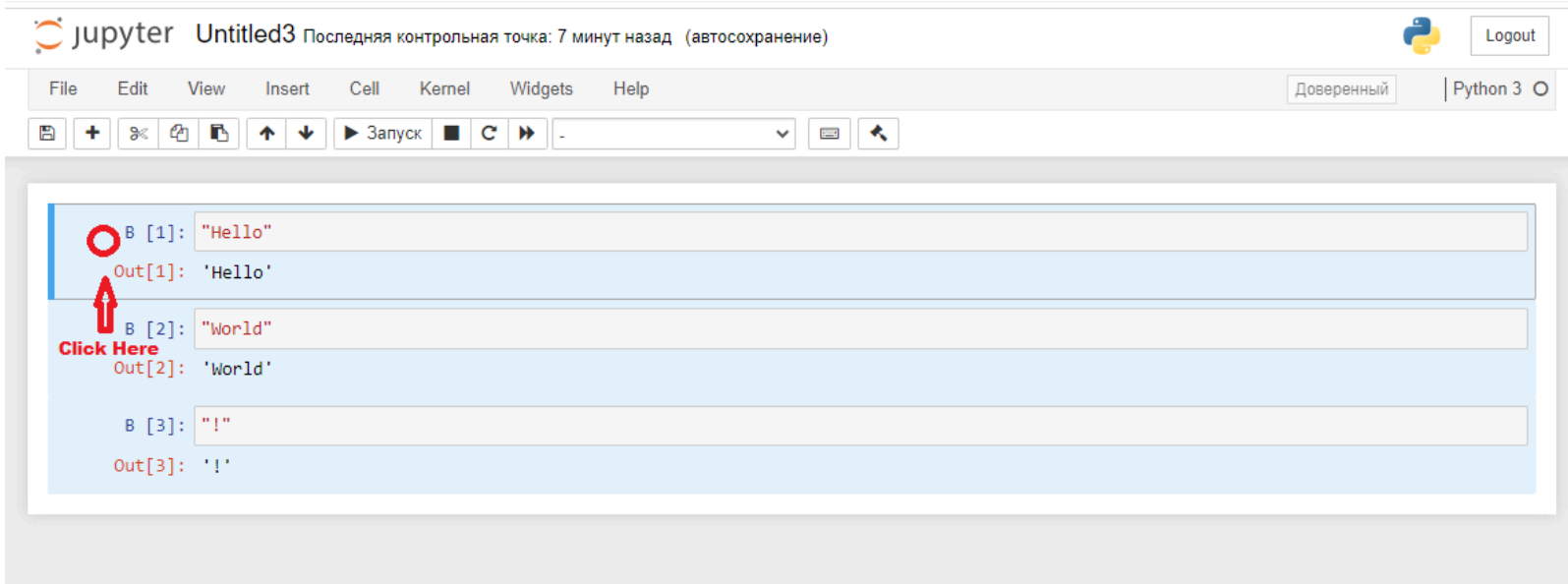
You can add a cell in a couple of ways:



- **Using the Toolbar:** Click the **+** button in the Jupyter Notebook toolbar. This will add a new cell below the currently selected cell.
- **From the menu bar:** Go to **Insert** → **Insert Cell Above** or **Insert Cell Below**.
- **Using Keyboard Shortcuts:**
 - Press **A** to add a new cell **above** the selected cell.
 - Press **B** to add a new cell **below** the selected cell.

Command Mode vs. Edit Mode

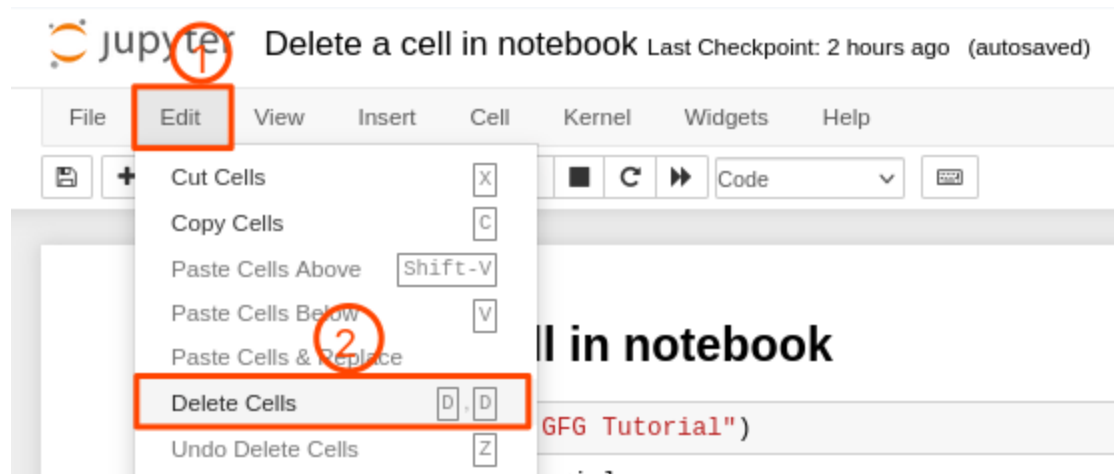
To select a cell *for commands*, click the area to the left of the entry field, *not inside it*.



- If you click inside the cell instead, it will enter Edit mode (highlighted with cursor inside), and commands such as **A** (insert cell above) or **B** (insert cell below) will not work.
- To exit Edit mode, press **Esc** (switched to Command Mode). To switch to Edit Mode, press **Enter**.

Deleting a cell

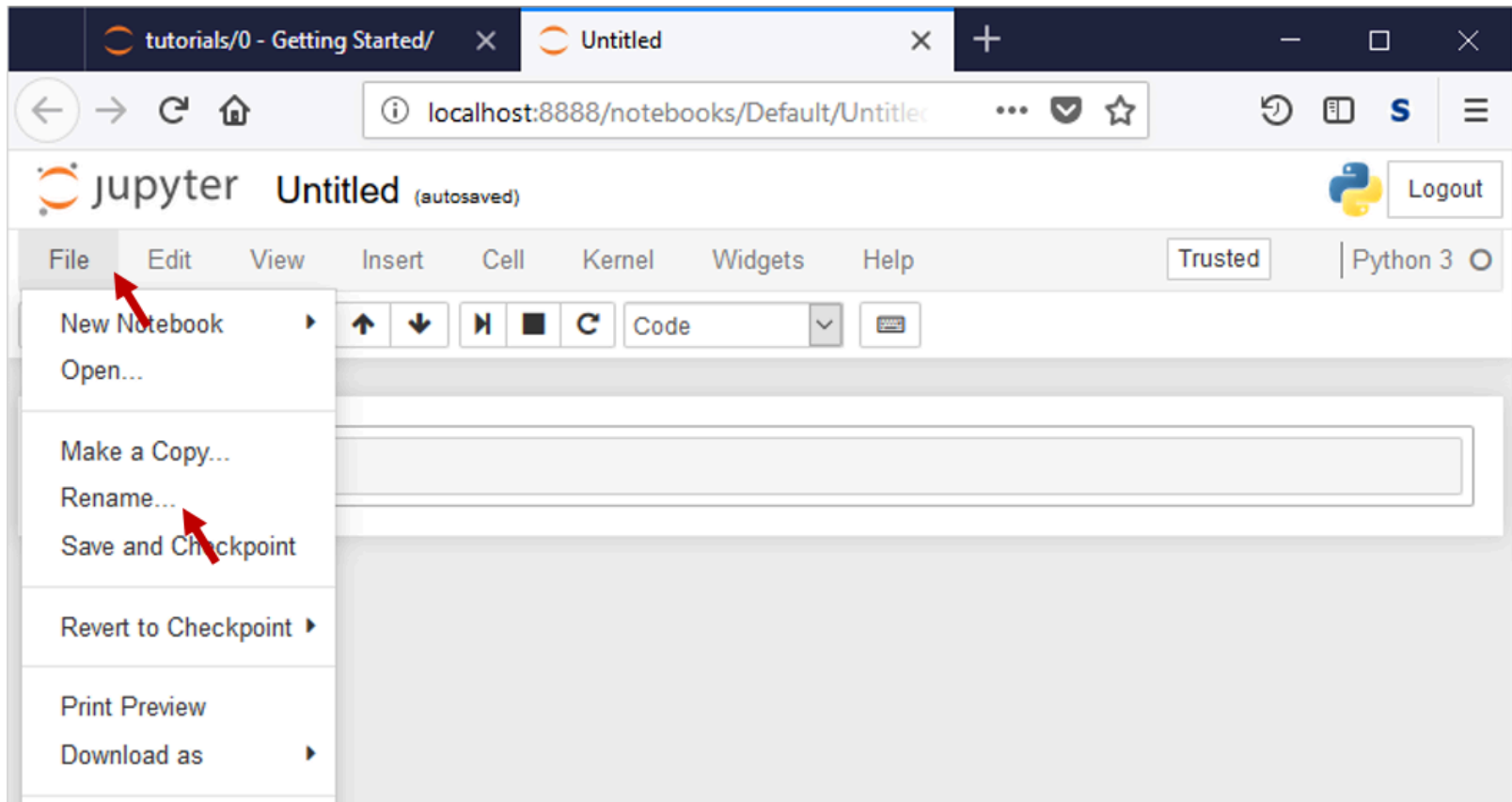
Select the cell you want to remove, then choose one of the following methods:



- Using the Toolbar: Click the scissors icon (✂) in the toolbar at the top.
- From the menu bar: Go to **Edit** → **Delete Cells**.
- Using Keyboard Shortcuts: Press **D** twice quickly (**D**, **D**).

Naming a notebook file

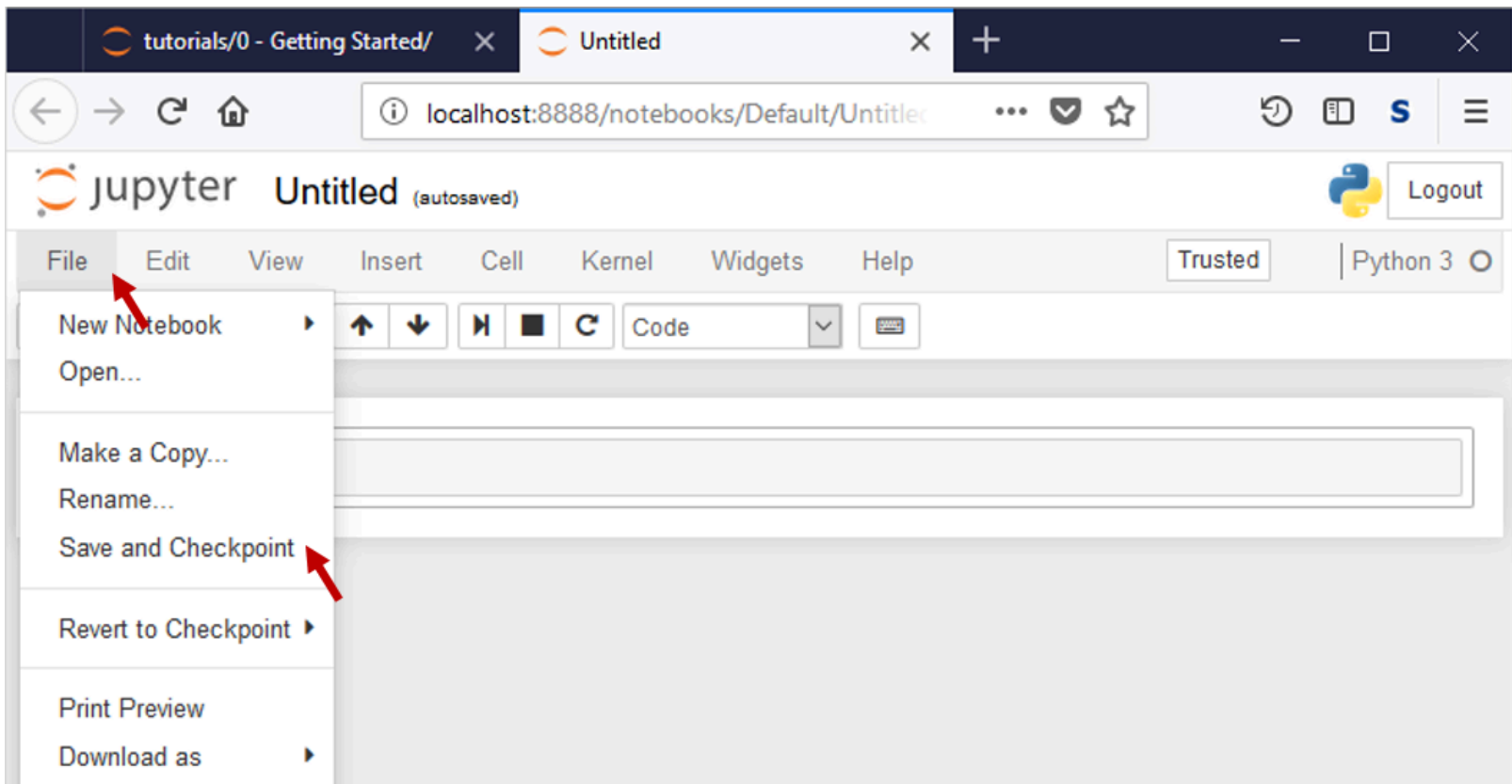
- By default, the new notebook will be stored in a file named `Untitled.ipynb`.
- ○ You can rename the file by clicking on `File` and `Rename` menu option at the top, as shown in the diagram below.



Saving a notebook file

- You can save the notebook by clicking on the **File** and **Save and Checkpoint** menu options. The notebook will be stored in a file with a **.ipynb** extension.

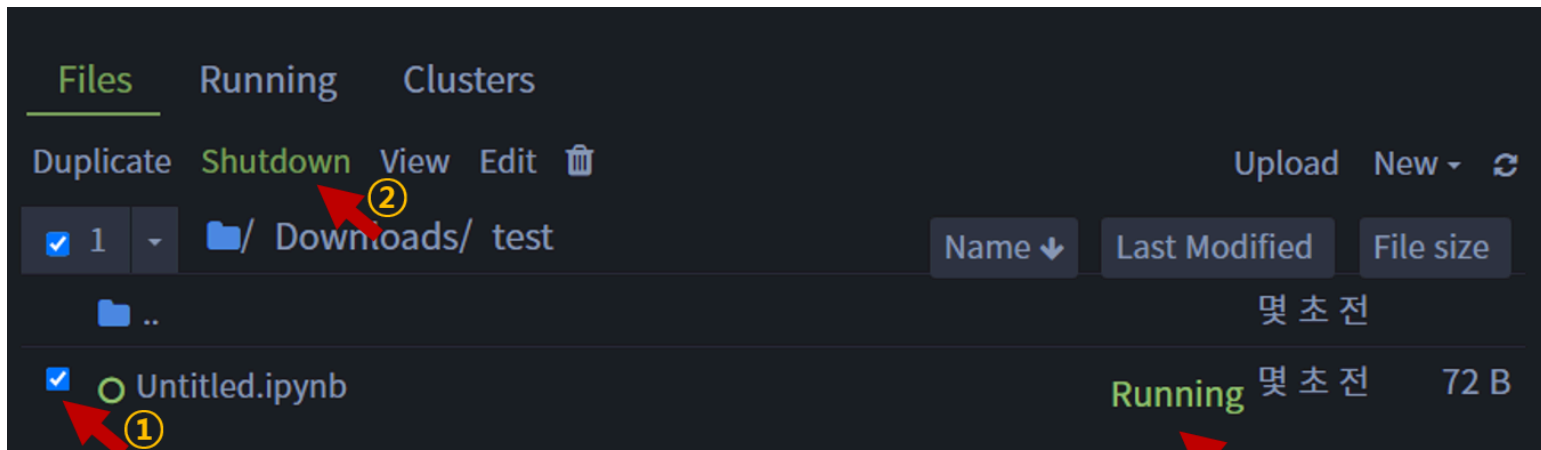
 The file is located in the directory where you created the notebook.



- After saving and closing this notebook file, you can open the notebook and re-run the program and the results you have saved any time.
- This powerful feature allows you to share your program and results as well as to reproduce the results generated by others.

Closing a notebook file

- Just closing the notebook is not enough; the python kernel is still running in the background.
- Please **shut down** the kernel by clicking the checkbox located left side of the notebook file (①), then click **Shutdown** (②).
 - After shutting down that kernel, **Running** sign goes off.

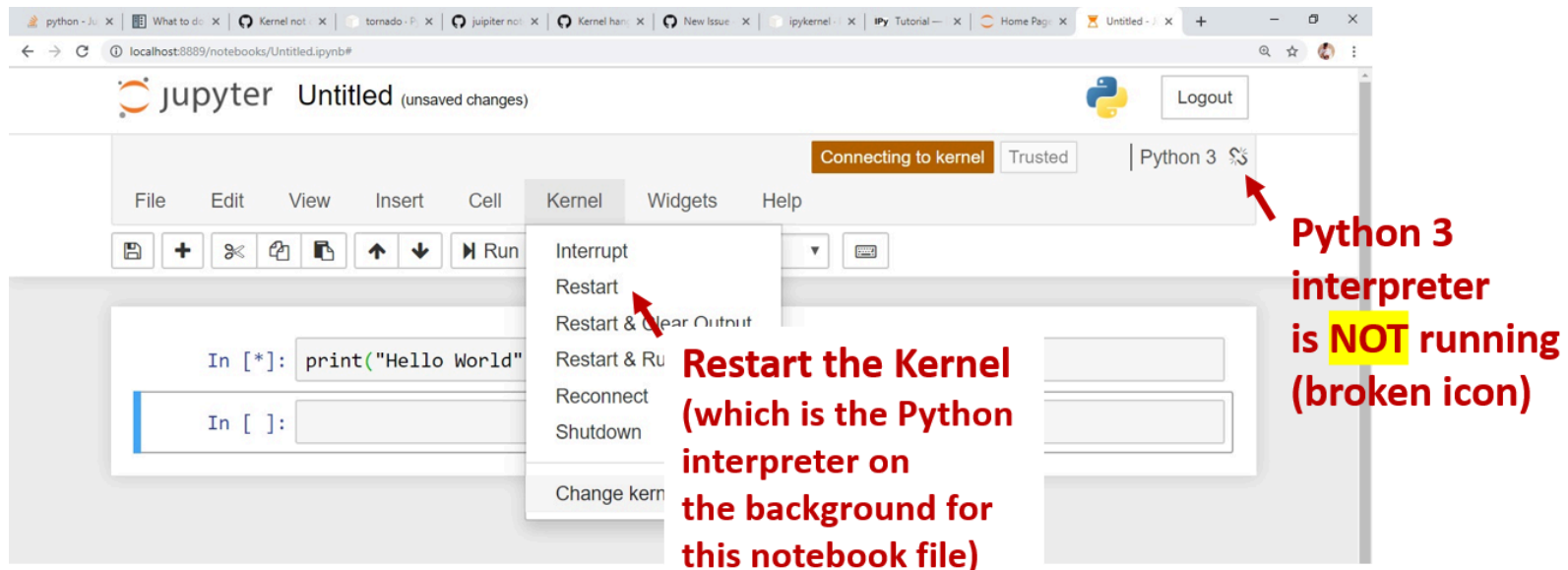


The python kernel
is still running.

FAQ

Q. My code is not working. The cell number is displayed as an asterisk [*].

- A1. Check that your notebook server is running on the Anaconda command prompt. If not, run it again, return to your web page, and reload the page.
- A2. If your notebook server is running well, please restart the kernel by clicking on **Kernel** and **Restart** menu option at the top. Then, check the **Python 3** interpreter icon shape has returned to a circle.



Advice: When you encounter an error...



For debugging,

1. Read the error message carefully! You will find hint for debugging.
2. Google the error messages. Some of the more than 10 million Python users had already encountered and fixed the error.

HOW TO BECOME
A WEB DEVELOPER:



1. Write broken code.



2. Google it.



3. Fix it.



Workload

The only way to learn Python, is by *writing Python... a lot*. So you are expected to put in effort.

   If you think you can learn Python by just listening to me, you are grossly overestimating my abilities.

Last words before we get to it.

- Work
- Make friends
- Fail often

Thank You.